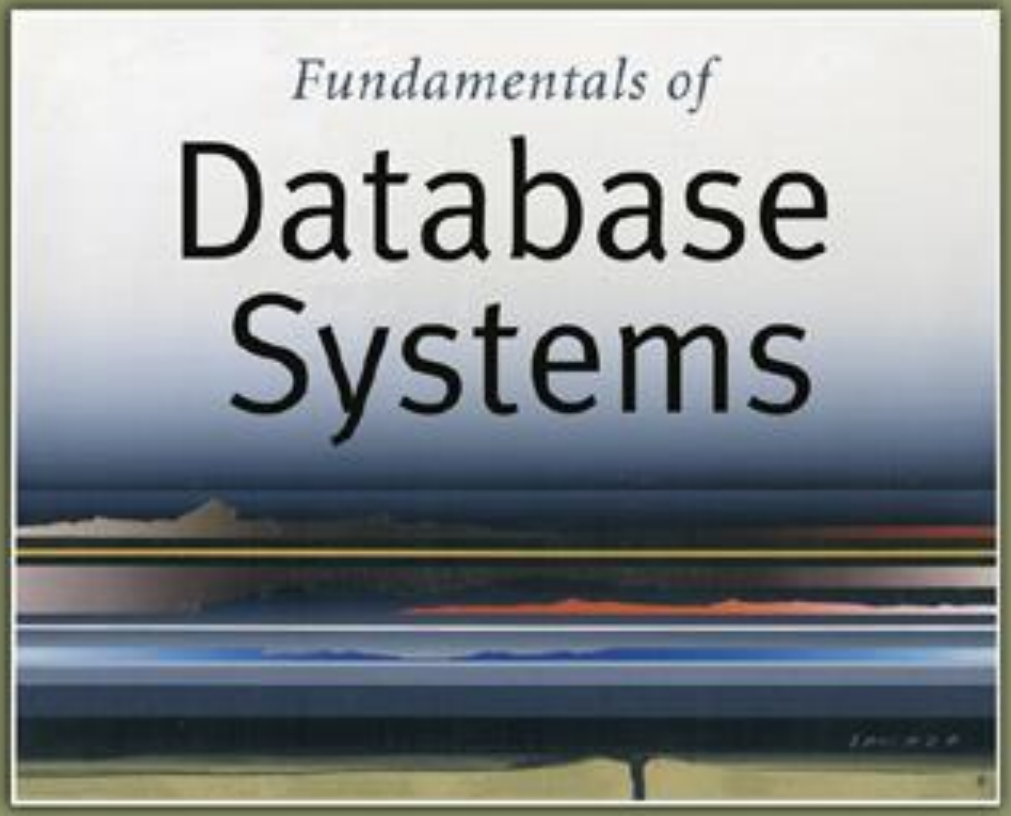
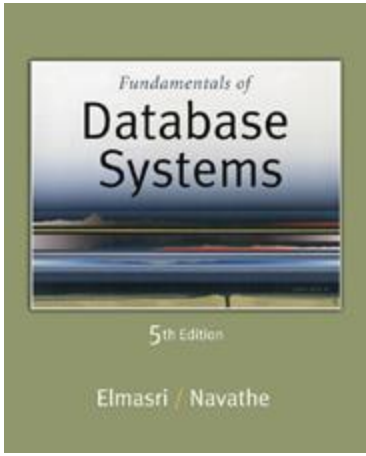


5th Edition

Elmasri / Navathe

Chapter 2

Database System Concepts and Architecture



Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Outline

- Data Models and Their Categories
- History of Data Models
- Schemas, Instances, and States
- Three-Schema Architecture
- DBMS Languages and Interfaces
- Database System Utilities and Tools
- Centralized and Client-Server Architectures
- Classification of DBMSs

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Data Models

- **Data Model:**
  - A set of concepts to describe the ***structure*** of a database, the ***operations*** for manipulating these structures, and certain ***constraints*** that the database should obey.
- **Data Model Structure and Constraints:**
  - Constructs are used to define the database structure
  - Constructs typically include ***elements*** (and their ***data types***) as well as groups of elements (e.g. ***entity, record, table***), and ***relationships*** among such groups
  - Constraints specify some restrictions on valid data; these constraints must be enforced at all times

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Data Models (continued)

- **Data Model Operations:**
  - These operations are used for specifying database *retrievals* and *updates* by referring to the constructs of the data model.
  - Operations on the data model may include
  - ***basic model operations*** (e.g. generic insert, delete, update) and
  - ***user-defined operations*** (e.g. compute_student_gpa, update_inventory)

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Categories of Data Models

- **Conceptual (high-level, semantic) data models:**
  - Provide concepts that are close to the way many users perceive data.
    - (Also called ***entity-based*** or ***object-based*** data models.)
- **Physical (low-level, internal) data models:**
  - Provide concepts that describe details of how data is stored in the computer. These are usually specified in an ad-hoc manner through DBMS design and administration manuals
- **Implementation (representational) data models:**
  - Provide concepts that fall between the above two, used by many commercial DBMS implementations (e.g. relational data models used in many commercial systems).

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Database Schemas versus Instances(State)

- Database Schema:
  - The ***description*** of a database.
  - Includes descriptions of the database structure, data types, and the constraints on the database.
- Schema Diagram:
  - An ***illustrative*** display of (most aspects of) a database schema.
- Schema Construct:
  - A ***component*** of the schema or an object within the schema, e.g., STUDENT, COURSE.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Schemas versus Instances

- Database State:
  - The actual data stored in a database at a ***particular moment in time***. This includes the collection of all the data in the database.
  - Also called database instance (or occurrence or snapshot).
    - The term *instance* is also applied to individual database components, e.g. *record instance, table instance, entity instance*
  - Initial Database State
  - Valid State:

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Database Schema vs. Database State

- Distinction
  - The ***database schema*** changes very infrequently.
  - The ***database state*** changes every time the database is updated.
- **Schema** is also called **intension**.
- **State** is also called **extension**

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Example of a Database Schema

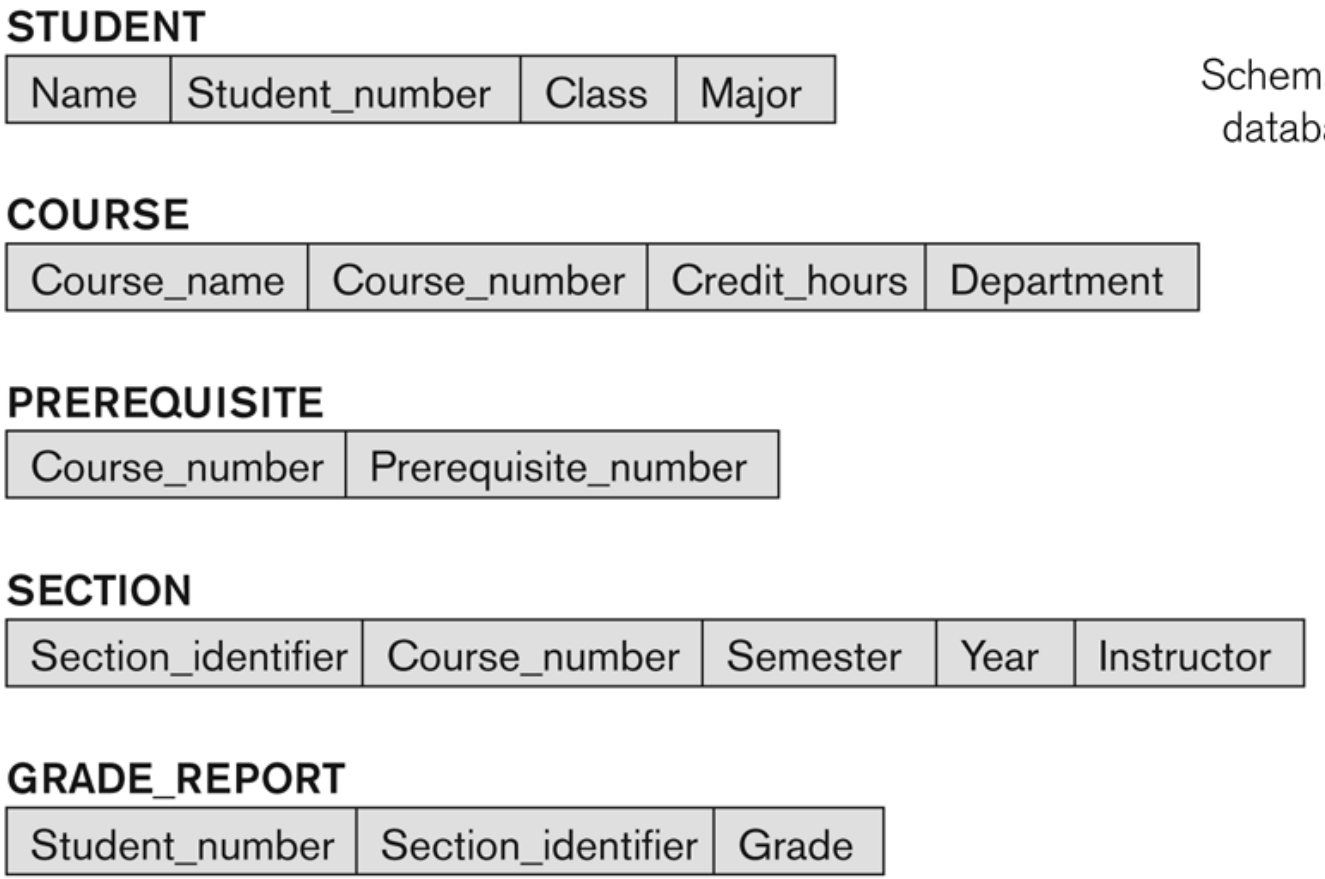


Figure 2.1
Schema diagram for the database in Figure 1.2.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Example of a database state

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	04	King
92	CS1310	Fall	04	Anderson
102	CS3320	Spring	05	Knuth
112	MATH2410	Fall	05	Chang
119	CS1310	Fall	05	Anderson
135	CS3380	Fall	05	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

Figure 1.2
A database that stores student and course information.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Three-Schema Architecture

- Proposed to support DBMS characteristics of:
  - **Program-data independence.**
  - Support of **multiple views** of the data.
- Not explicitly used in commercial DBMS products, but has been useful in explaining database system organization

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

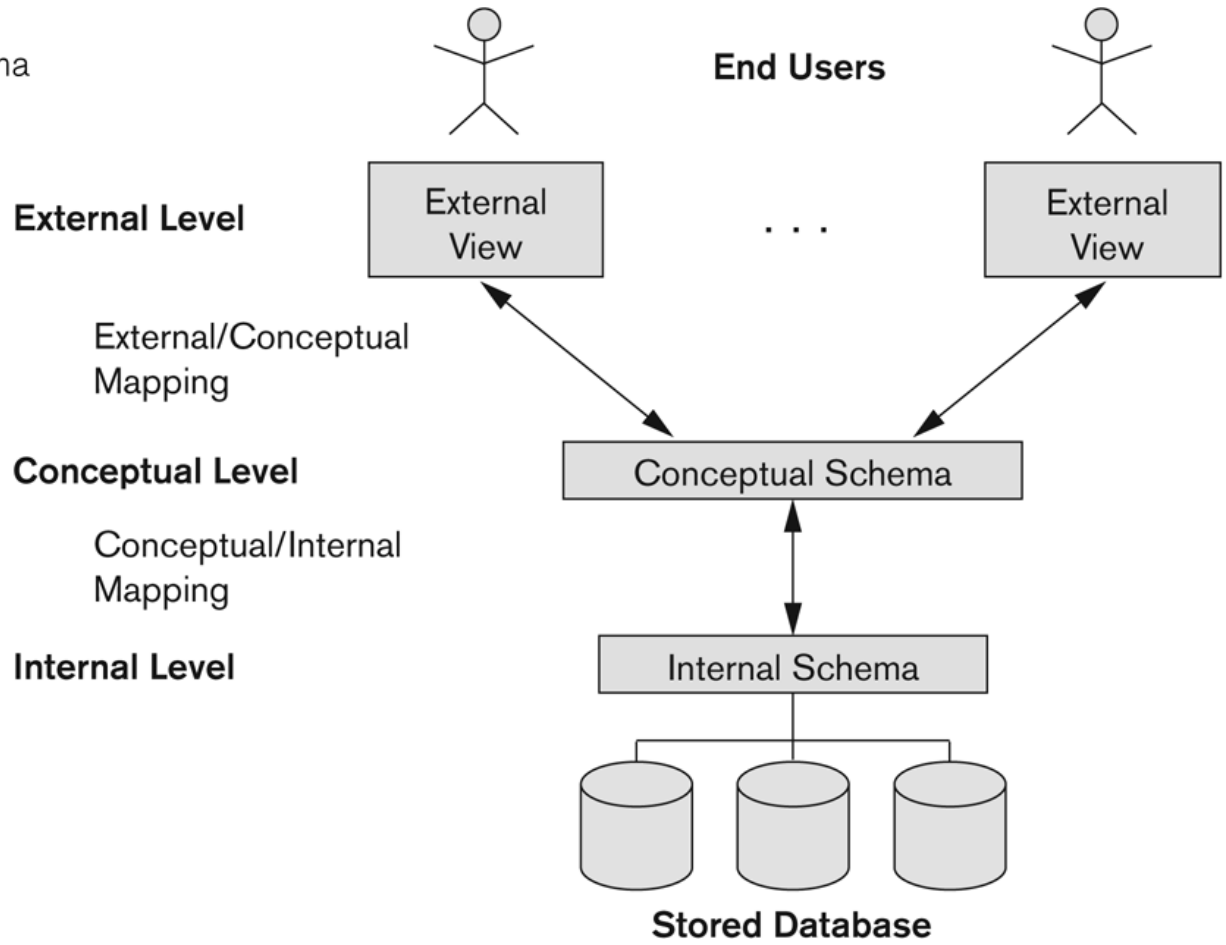
Three-Schema Architecture

- Defines DBMS schemas at ***three*** levels:
  - **Internal schema** at the internal level to describe physical storage structures and access paths (e.g indexes).
    - Typically uses a **physical** data model.
  - **Conceptual schema** at the conceptual level to describe the structure and constraints for the whole database for a community of users.
    - Uses a **conceptual** or an **implementation** data model.
  - **External schemas** at the external level to describe the various user views.
    - Usually uses the same data model as the conceptual schema.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

The three-schema architecture

Figure 2.2
The three-schema architecture.



Data Independence

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Data Independence

- **Data Independence:**
- The capacity to change the one level of schema without having to change another level schemas

Data Independence

Logical

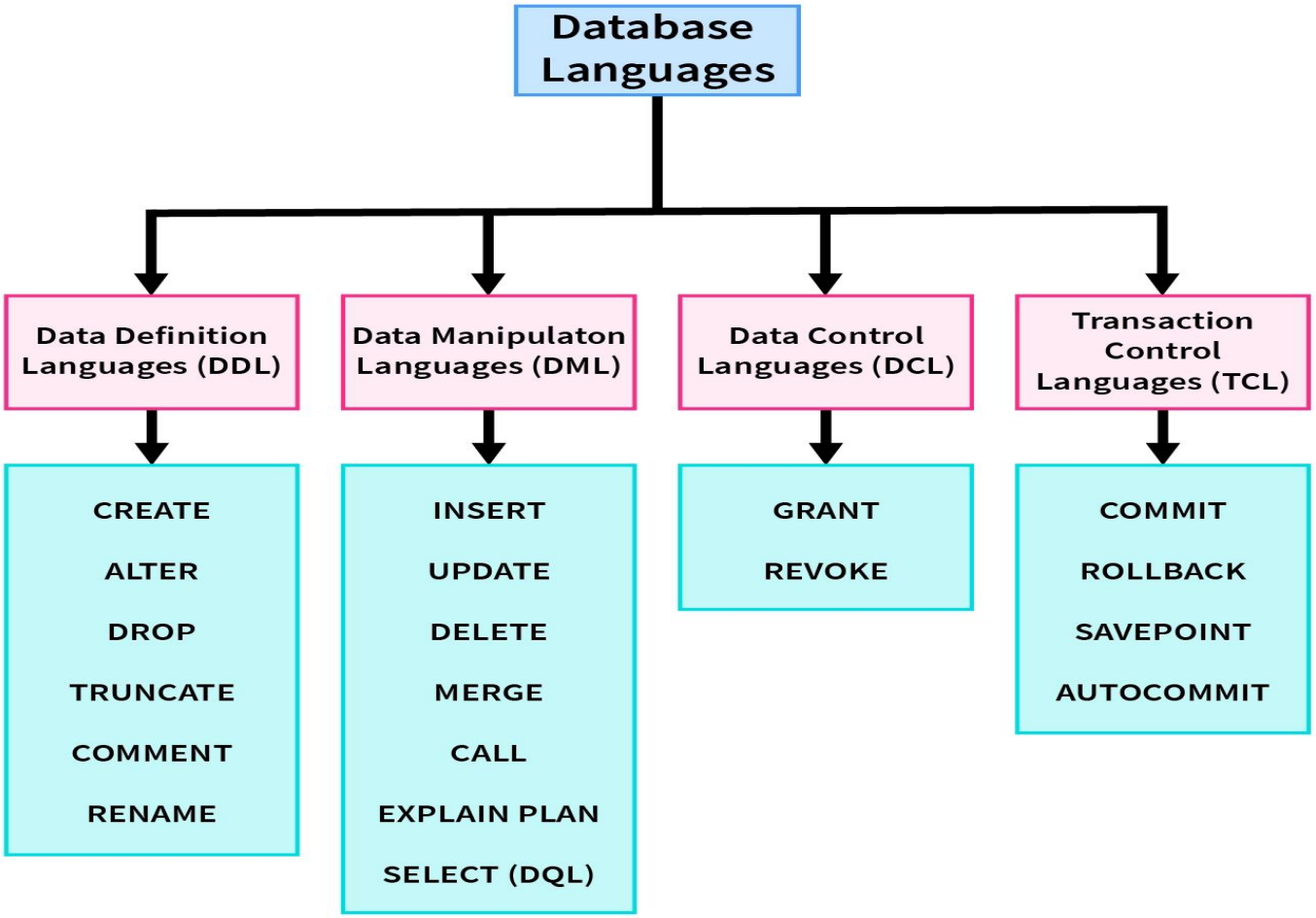
The capacity to change the conceptual schema without having to change the external schemas

Physical

The capacity to change the internal schema without having to change the conceptual schema

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

DBMS Languages



Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

DBMS Languages

- **Data Definition Language (DDL):**
  - In many DBMSs, the DDL is also used to define internal and external schemas (views).
  - In some DBMSs, separate **storage definition language (SDL)** and **view definition language (VDL)** are used to define internal and external schemas.
    - SDL is typically realized via DBMS commands provided to the DBA and database designers

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

DBMS Languages

- **Data Manipulation Language (DML):**
  - Used to specify database retrievals and updates
  - DML commands (data sublanguage) can be *embedded* in a general-purpose programming language (host language), such as COBOL, C, C++, or Java.
    - A library of functions can also be provided to access the DBMS from a programming language
  - Alternatively, stand-alone DML commands can be applied directly (called a *query language*).

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

DBMS Interfaces

- Stand-alone query language interfaces
  - Example: Entering SQL queries at the DBMS interactive SQL interface (e.g. SQL*Plus in ORACLE)
- Programmer interfaces for embedding DML in programming languages
- User-friendly interfaces
  - Menu-based, forms-based, graphics-based, etc.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Other DBMS Interfaces

- Speech as Input and Output
- Web Browser as an interface
- Parametric interfaces, e.g., bank tellers using function keys.
- Interfaces for the DBA:
  - Creating user accounts, granting authorizations
  - Setting system parameters
  - Changing schemas or access paths

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Database System Utilities

- To perform certain functions such as:
  - Loading data stored in files into a database. Includes data conversion tools.
  - Backing up the database periodically on tape.
  - Reorganizing database file structures.
  - Report generation utilities.
  - Performance monitoring utilities.
  - Other functions, such as sorting, user monitoring, data compression, etc.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Other Tools

- Data dictionary / repository:
  - Used to store schema descriptions and other information such as design decisions, application program descriptions, user information, usage standards, etc.
  - **Active data dictionary** is accessed by DBMS software and users/DBA.
  - **Passive data dictionary** is accessed by users/DBA only.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Centralized and Client-Server DBMS Architectures

- Centralized DBMS:
  - Combines everything into single system including-DBMS software, hardware, application programs, and user interface processing software.
  - User can still connect through a remote terminal – however, all processing is done at centralized site.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

A Physical Centralized Architecture

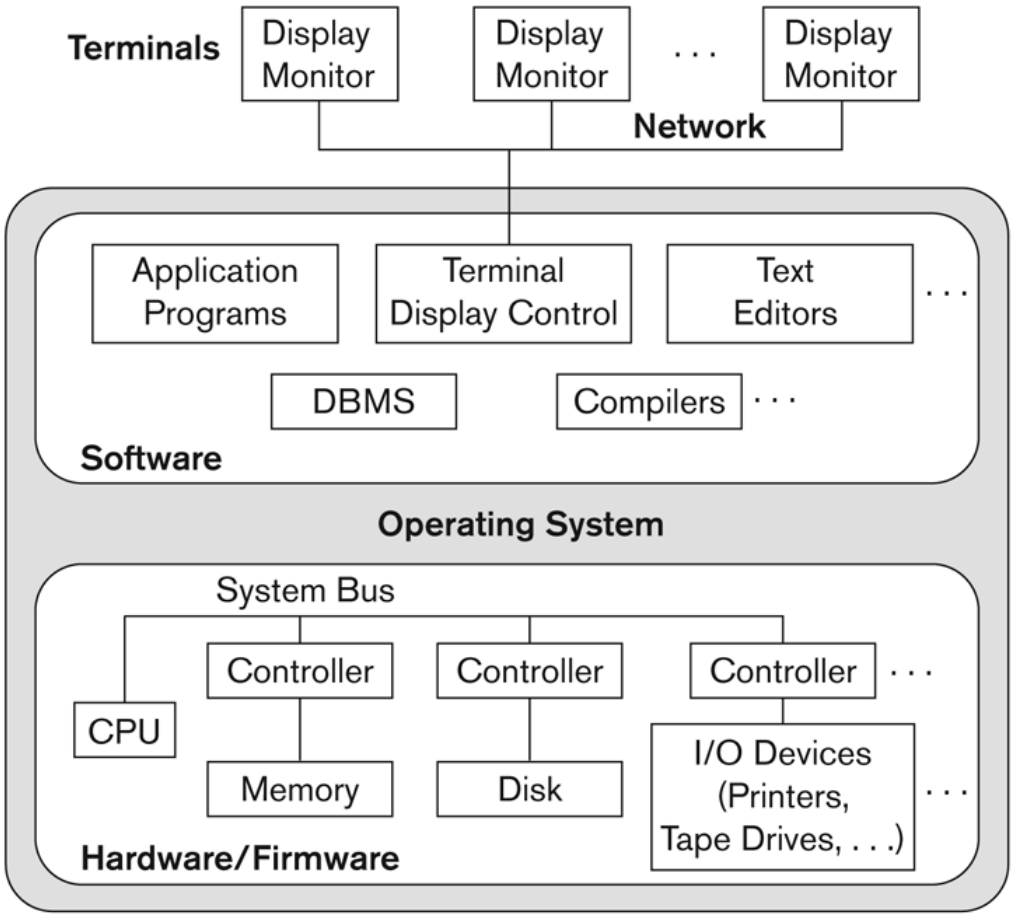


Figure 2.4
A physical centralized architecture.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

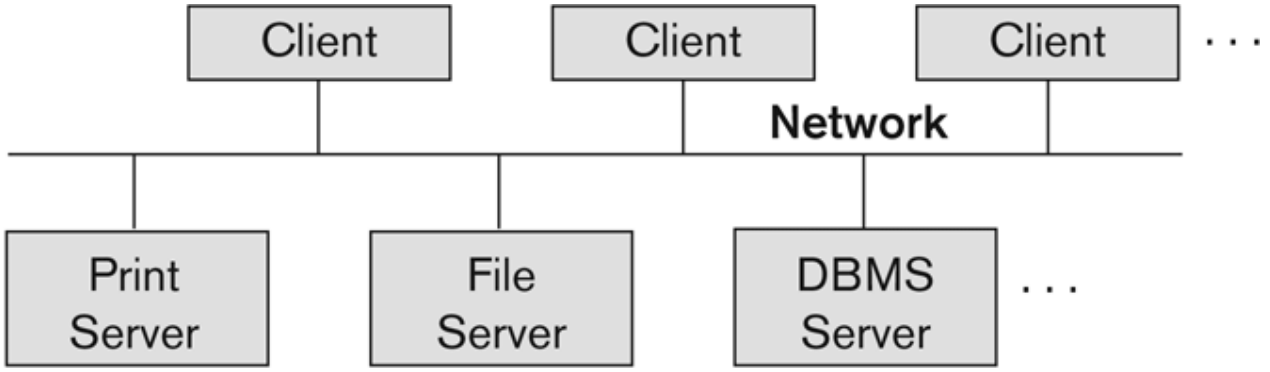
Basic 2-tier Client-Server Architectures

- Specialized Servers with Specialized functions
  - Print server
  - File server
  - DBMS server
  - Web server
  - Email server
- Clients can access the specialized servers as needed

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Logical two-tier client server architecture

Figure 2.5
Logical two-tier client/server architecture.



Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Clients

- Provide appropriate interfaces through a client software module to access and utilize the various server resources.
- Clients may be diskless machines or PCs or Workstations with disks with only the client software installed.
- Connected to the servers via some form of a network.
  - (LAN: local area network, wireless network, etc.)

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

DBMS Server

- Provides database query and transaction services to the clients
- Relational DBMS servers are often called SQL servers, query servers, or transaction servers
- Applications running on clients utilize an Application Program Interface (**API**) to access server databases via standard interface such as:
  - ODBC: Open Database Connectivity standard
  - JDBC: for Java programming access
- Client and server must install appropriate client module and server module software for ODBC or JDBC
- See Chapter 9

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Two Tier Client-Server Architecture

- A client program may connect to several DBMSs, sometimes called the data sources.
- In general, data sources can be files or other non-DBMS software that manages data.
- Other variations of clients are possible: e.g., in some object DBMSs, more functionality is transferred to clients including data dictionary functions, optimization and recovery across multiple servers, etc.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

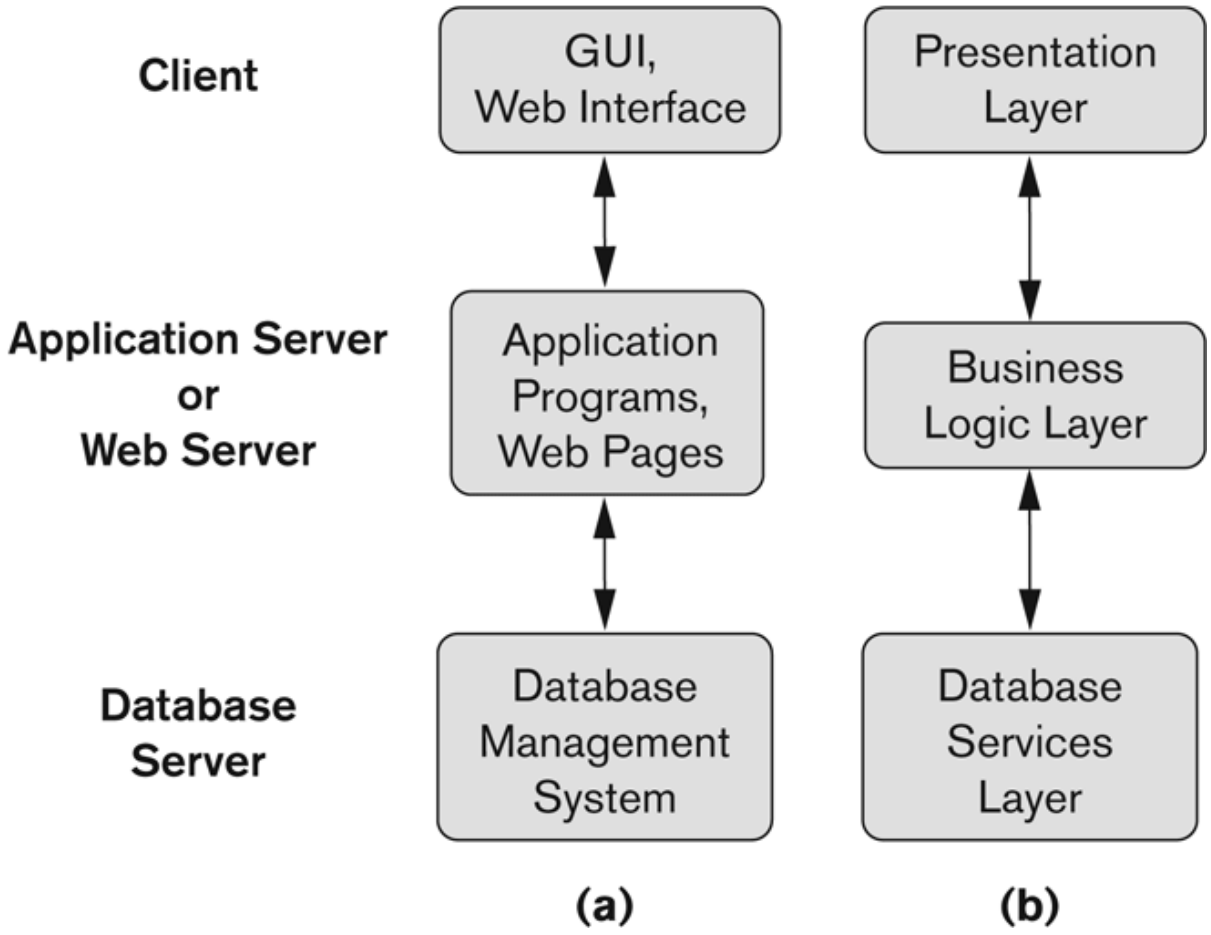
Three Tier Client-Server Architecture

- Common for Web applications
- Intermediate Layer called Application Server or Web Server:
  - Stores the web connectivity software and the business logic part of the application used to access the corresponding data from the database server
  - Acts like a conduit for sending partially processed data between the database server and the client.
- Three-tier Architecture Can Enhance Security:
  - Database server only accessible via middle tier
  - Clients cannot directly access database server

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Three-tier client-server architecture

Figure 2.7
Logical three-tier client/server architecture, with a couple of commonly used nomenclatures.



Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Classification of DBMSs

- Based on the data model used
  - Traditional: Relational, Network, Hierarchical.
  - Emerging: Object-oriented, Object-relational.
- Other classifications
  - Single-user (typically used with personal computers)
    vs. multi-user (most DBMSs).
  - Centralized (uses a single computer with one database)
    vs. distributed (uses multiple computers, multiple databases)

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Summary

- Data Models and Their Categories
- History of Data Models
- Schemas, Instances, and States
- Three-Schema Architecture
- Data Independence
- DBMS Languages and Interfaces
- Database System Utilities and Tools
- Centralized and Client-Server Architectures
- Classification of DBMSs

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe